

# Part 5 Report: Testable Design, Stubbing, Bad Testable Design, and Mocking

---

**Project:** TuxGuitar (Open Source Tablature Editor)

**Course:** SWE 261P

**Member:** Xiyao Li & Ping Lu

**Date:** February 20, 2026

**Base repository link:** <https://github.com/helge17/tuxguitar>

**Forked repository link:** <https://github.com/lupingblaine/SWE-261P-tuxguitar>

## 1. Testable Design: aspects and goals

A testable design should make behavior easy to verify in isolation and in a repeatable way.

Key aspects and goals:

1. Dependency control: dependencies should be injectable, not hard-coded, so tests can replace them with stubs, fakes, or mocks.
2. Low coupling: each class should have clear responsibilities and minimal knowledge of other components.
3. Deterministic behavior: avoid hidden global state and non-deterministic side effects (threads, time, I/O) in core logic.
4. Observable outcomes: behavior should be verifiable through return values, state changes, or collaborator interactions.
5. Fast and focused tests: unit tests should run without heavy external systems.

## 2. Stubbing

### 2.1 Existing stubbing example in codebase

Existing example:

- `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestLetRing.java`[Click](#)

How stubbing is used:

- The test defines `MidiSequencerImplForTest` extends `MidiSequencerImpl`.
- It overrides `addEvent(MidiEvent event)` and stores events in a list instead of using the real sequencer event pipeline.

Why this is used:

- The test needs direct control and inspection of generated MIDI events.
- Overriding avoids relying on the full runtime behavior of real sequencer internals.
- This keeps the test focused on parser output.

## 2.2 New stubbing implementation in a test

Implemented file:

- `common/TuxGuitar-lib/src/test/java/app/tuxguitar/thread/TestTGThreadManagerStubbing.java` [Click](#)

What was done:

1. Created a stub implementation of `TGThreadHandler` (`StubThreadHandler`) in the test.
2. Stubbed `start(TGThreadPriority priority, Runnable runnable)` to record calls by priority.
3. Injected this stub into `TGThreadManager` using `setThreadHandler(...)`.
4. Called:
  - `manager.start(...)` (default normal priority)
  - `manager.start(TGThreadPriority.HIGH, ...)`
5. Verified:
  - the stubbed method was used instead of real thread handlers;
  - total calls and per-priority call counts match expectations.

This satisfies the requirement to stub an existing method used in a test and run the test with the stubbed method instead of the real implementation.

Run test:

```
./mvnw -f common/TuxGuitar-lib/pom.xml -Dtest=TestTGThreadManagerStubbing
test
```

## 3. Bad testable design

### 3.1 Problematic design identified

Problematic implementation:

- `common/TuxGuitar-lib/src/main/java/app/tuxguitar/player/base/MidiSynthesizerProxy.java` (original behavior)

Original code (before refactor):

```
public MidiChannel openChannel(int channelId) throws MidiPlayerException {
    if (this.midiSynthesizer != null) {
        return this.midiSynthesizer.openChannel(channelId);
    }
    return null;
}
```

Issue:

- When `midiSynthesizer` is not configured, methods silently return defaults (`null/false`) or no-op.
- There is no explicit fail-fast behavior for misconfiguration.

Why this is bad for testability:

- Misconfiguration behavior is weakly observable and easy to miss.
- We cannot write a precise test that enforces “missing dependency must fail fast”.

### 3.2 Recommended fix

Add a testable strict mode:

- Keep backward-compatible default behavior (`strictMode = false`).
- Add strict mode so missing synthesizer throws `MidiPlayerException`.

### 3.3 Implemented new version

Changed file:

- `common/TuxGuitar-lib/src/main/java/app/tuxguitar/player/base/MidiSynthesizerProxy.java`

New functionality added:

- `setStrictMode(boolean strictMode) / isStrictMode()`
- In strict mode, `openChannel`, `closeChannel`, `isChannelOpen`, and `isBusy` throw `MidiPlayerException` when `midiSynthesizer` is not set.
- In non-strict mode, old behavior is preserved.

### 3.4 Test for newly testable behavior

Implemented file:

- `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiSynthesizerProxyTestability.java` [Click](#)

Test behavior:

1. Enables strict mode with `proxy.setStrictMode(true)`.
2. Verifies `openChannel(...)` throws when synthesizer is missing.
3. Verifies `closeChannel(...)` throws when synthesizer is missing.
4. Verifies `isChannelOpen(...)` and `isBusy()` also throw in the same state.

Run test:

```
./mvnw -f common/TuxGuitar-lib/pom.xml -
Dtest=TestMidiSynthesizerProxyTestability test
```

## 4. Mocking

## 4.1 Mocking and utility

Mocking replaces real collaborators with programmable test doubles and enables behavior verification.

Utility:

1. Verify whether collaborator methods were called.
2. Verify call count and call arguments.
3. Verify delegation and fan-out behavior without depending on real implementations.

## 4.2 Feature A to mock: `MidiSynthesizerProxy` delegation

Feature:

- `MidiSynthesizerProxy` forwards calls to its internal `MidiSynthesizer` when one is set.

Why mocking helps:

- Without mocking, we cannot precisely verify that the proxy forwarded calls with exact arguments.
- With Mockito, we can verify interaction behavior (`openChannel`, `closeChannel`, `isChannelOpen`, `isBusy`) and confirm default behavior when no synthesizer is set.

Implemented test:

- `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiSynthesizerProxyMocking.java` [Click](#)

Run test:

```
./mvnw -f common/TuxGuitar-lib/pom.xml -
Dtest=TestMidiSynthesizerProxyMocking test
```

## 4.3 Feature B to mock: `MidiTransmitter` fan-out and receiver management

Feature:

- `MidiTransmitter` broadcasts events to registered `MidiReceivers` and manages receivers by ID.

Why mocking helps:

- Without mocking, it is hard to prove exactly which receivers received events.
- With Mockito, we can verify fan-out behavior, duplicate-ID handling, and removal behavior.

Implemented test:

- `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiTransmitterMocking.java` [Click](#)

Run test:

```
./mvnw -f common/TuxGuitar-lib/pom.xml -Dtest=TestMidiTransmitterMocking
test
```

## 4.4 Mockito test dependency

Added in:

- `common/TuxGuitar-lib/pom.xml`

Dependency:

- `org.mockito:mockito-core:4.11.0` (test scope)

## 5. New files added in this task

1. `Part5Report.md`
2. `common/TuxGuitar-lib/src/test/java/app/tuxguitar/thread/TestTGThreadManagerStubbing.java`
3. `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiSynthesizerProxyTestability.java`
4. `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiSynthesizerProxyMocking.java`
5. `common/TuxGuitar-lib/src/test/java/app/tuxguitar/player/base/TestMidiTransmitterMocking.java`

## 6. Existing files modified in this task

1. `common/TuxGuitar-lib/src/main/java/app/tuxguitar/player/base/MidiSynthesizerProxy.java`
2. `common/TuxGuitar-lib/pom.xml`